WEB TECHNOLOGIES · ALL MINI PROJECTS

All my mini projects in one presentation



Aykhan Dadashov · Web Technologies · ISMA

01 Overview of my mini projects

Project	Course	What I practised	Live
Rock Paper Scissors	Foundations	functions, conditionals, DOM events	ayxan1110.github.io/odin-rock-paper-scissors/
Etch-a-Sketch	Foundations	loops, DOM creation, mouse events	ayxan1110.github.io/odin-etch-a-sketch/
Calculator	Foundations	state, keyboard support, no eval()	ayxan1110.github.io/odin-calculator/
Sign-up Form	Intermediate HTML & CSS	forms, validation, Flexbox	ayxan1110.github.io/odin-sign-up-form/
Admin Dashboard	Intermediate HTML & CSS	CSS Grid, SVG icons	ayxan1110.github.io/odin-admin-dashboard/
Library	JavaScript	constructors, prototypes, dialog	ayxan1110.github.io/odin-library/
Tic Tac Toe	JavaScript	factory functions, module pattern	ayxan1110.github.io/odin-tic-tac-toe/
Homepage	Advanced HTML & CSS	responsive design, accessibility	ayxan1110.github.io/odin-homepage/
CareerBridge	Project idea (11.11)	AI agents, product pitch, prototype	ayxan1110.github.io/careerbridge-prototype/

PART 1

Foundations (Open Course)

JavaScript Basics: Rock Paper Scissors, Etch-a-Sketch, Calculator

02 Lessons in JavaScript Basics

1 Variables and Operators	2 Installing Node.js	3 Data Types and Conditionals	4 JavaScript Developer Tools	5 Function Basics
6 Problem Solving	7 Understanding Errors	8 **Project: Rock Paper Scissors**	9 Clean Code	10 Loops and Arrays
11 DOM Manipulation and Events	12 Revisiting Rock Paper Scissors	13 **Project: Etch-a-Sketch**	14 Object Basics	15 **Project: Calculator**

03 Variables, operators, data types, conditionals

let and const – const cannot be reassigned, let can; we do not use var

Operators – + - * / % ** and ++ / --, careful with "5" + 2 = "52"

8 data types – string, number, bigint, boolean, undefined, null, symbol, object

Comparison – always use === (strict) instead of ==

Conditionals – if / else if / else, switch, the ternary operator

Logical operators – && (and), || (or), ! (not)

basics.js

```
const name = "Aykhan";   // cannot change
let score = 0;           // can change
score++;                 // 1

"5" + 2;     // "52"  (string)
"5" * 2;     // 10    (number)
5 === "5";   // false (strict)

if (score >= 5) {
  console.log("You win!");
} else if (score === 0) {
  console.log("Start playing");
} else {
  console.log("Keep going");
}

const text = score > 0 ? "yes" : "no";
```

04

Tools, functions, problem solving, errors

Node.js and DevTools

Node runs JavaScript outside the browser. In Chrome DevTools I used the Console, the Elements panel and breakpoints in Sources to go through code line by line.

Function basics

Parameters and arguments, return values, function declarations, function expressions and arrow functions. Function scope keeps variables inside.

Problem solving

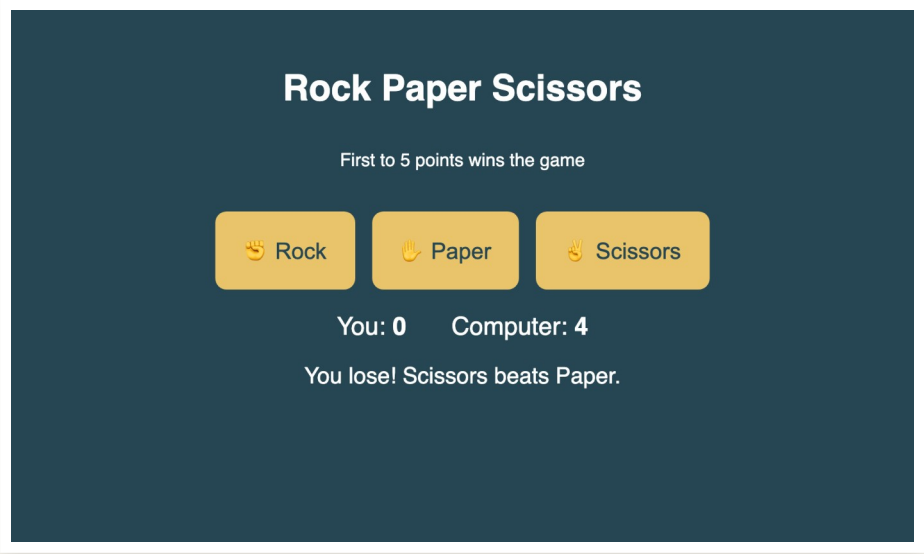
First understand the problem, then plan, write pseudocode and divide it into small steps. Then turn every step into code.

Understanding errors

Read the error message and the line number. ReferenceError: variable not defined. SyntaxError: wrong code. TypeError: wrong type of value.

05

Project: Rock Paper Scissors



- Computer choice with Math.random()
- playRound() compares the two choices
- Score for human and computer
- First to 5 points wins the game
- UI with buttons (Revisiting RPS lesson)

Live ayxan1110.github.io/odin-rock-paper-scissors/

Code github.com/Ayxan1110/odin-rock-paper-scissors

06

Clean code, loops and arrays

Clean code – good names (camelCase), consistent indentation, comments explain why

Arrays – ordered lists, index starts at 0; push, pop, slice, splice, includes

Loops – for, while, and for...of to repeat code

Array methods – map, filter, reduce, forEach – shorter than loops

Used in my projects – for loop in Etch-a-Sketch, forEach in RPS, includes() in the Calculator

arrays.js

```
const numbers = [1, 2, 3, 4, 5];

for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}

// new array, every number doubled
numbers.map((n) => n * 2);
// [2, 4, 6, 8, 10]

// only even numbers
numbers.filter((n) => n % 2 === 0);
// [2, 4]

// add everything together
numbers.reduce((sum, n) => sum + n, 0);
// 15
```

07

DOM manipulation and events

DOM – the page as a tree of nodes that JavaScript can change

Select – querySelector() and querySelectorAll()

Create and add – createElement(), appendChild(), append()

Change – textContent, classList, style, setAttribute()

Events – addEventListener("click", …), the event object, bubbling

dom.js

```
const container =
  document.querySelector("#container");

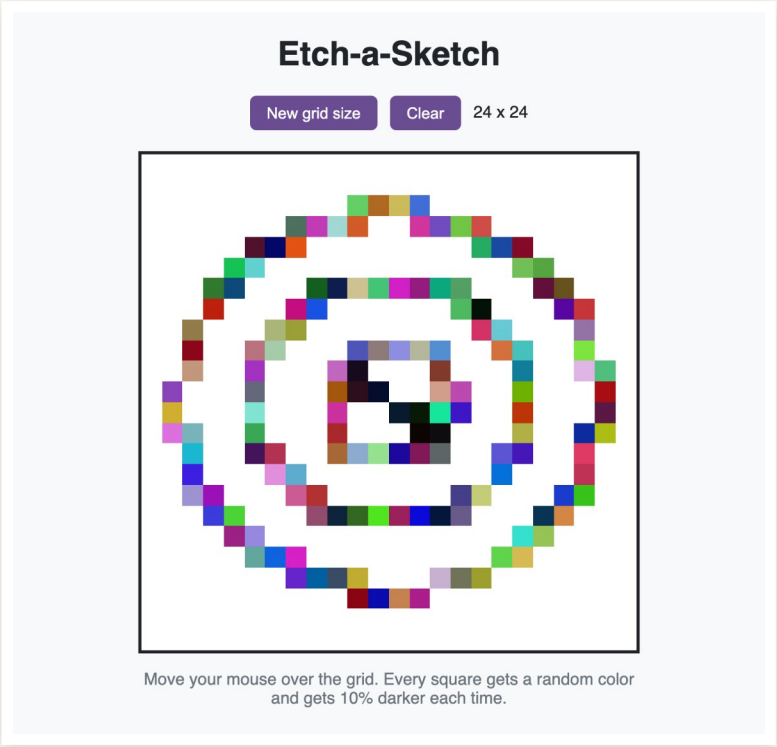
const button =
document.createElement("button");
button.textContent = "Click me";
button.classList.add("btn");
container.appendChild(button);

button.addEventListener("click", (event) => {
  console.log(event.target); // the button
  button.style.backgroundColor = "gold";
});

// many elements at once
document.querySelectorAll(".square")
  .forEach((sq) => sq.remove());
```

08

Project: Etch-a-Sketch



- 16 x 16 grid made with a JavaScript loop
- Flexbox with flex-wrap keeps it in a 480px box
- Squares change color on mouseenter
- Button with prompt() for a new size (max 100)
- Extra: random RGB and 10% darker every pass

Live ayxan1110.github.io/odin-etch-a-sketch/

Code github.com/Ayxan1110/odin-etch-a-sketch

09

Object basics

Objects – store data as key: value pairs

Add, change, delete – user.age = 23, delete user.age

Check a key – "name" in user, or loop with for...in

Reference – copying an object copies the reference, not the data

Array of objects – very common, e.g. a list of users or books

objects.js

```
const user = {
  name: "Aykhan",
  city: "Riga",
};

user.age = 23;           // add
user.city = "Baku";      // change
delete user.age;         // remove
"name" in user;          // true

const copy = user;       // same object!
copy.name = "Test";
user.name;               // "Test"

const users = [{ name: "A", age: 20 },
               { name: "B", age: 31 }];
users.find((u) => u.age > 30); // B
```

10 Project: Calculator



- add, subtract, multiply, divide + operate()
- One pair at a time: 12 + 7 − shows 19
- Rounds long decimals to fit the screen
- Error message when dividing by 0
- Extra: decimal, backspace, keyboard

Live ayxan1110.github.io/odin-calculator/

Code github.com/Ayxan1110/odin-calculator

11

Calculator: how the logic works

`firstNumber`

the number before the operator

`operator`

`+` `−` `×` `÷` that was pressed

`currentInput`

the number on the display

Press	firstNumber	operator	currentInput	Display
1, 2	empty	empty	12	12
+	12	+	12	12
7	12	+	7	7
−	19	−	19	19
1, =	empty	empty	18	18

PART 2

Intermediate HTML and CSS

Sign-up Form, Admin Dashboard and my profile page

12

Intermediate HTML and CSS: what I covered

Intermediate HTML

- Introduction
- SVG
- Tables

Intermediate CSS

- Default Styles
- CSS Units
- More Text Styles
- More CSS Properties
- Advanced Selectors
- Positioning
- CSS Functions
- Custom Properties
- Browser Compatibility
- Frameworks and Preprocessors

Forms

- Form Basics
- Form Validation
- **Project: Sign-up Form**

Grid

- Introduction to Grid
- Creating a Grid
- Positioning Grid Elements
- Advanced Grid Properties
- Using Flexbox and Grid
- **Project: Admin Dashboard**

13

SVG and tables

SVG – vector images written in XML, they stay sharp at any size

viewBox – the coordinate system of the drawing, so it can scale

Shapes – <circle>, <rect>, <path>; colors with fill and stroke

Tables – only for tabular data, not for page layout

Table tags – <table>, <tr>, <th>, <td>, <thead>, <tbody>, colspan

intermediate.html

```
<!-- a simple SVG icon -->
<svg viewBox="0 0 100 100" width="80">
  <circle cx="50" cy="50" r="40"
          fill="gold" stroke="black" />
</svg>

<!-- a simple table -->
<table>
  <thead>
    <tr><th>Subject</th><th>Mark</th></tr>
  </thead>
  <tbody>
    <tr><td>Web Tech</td><td>9</td></tr>
    <tr><td>Programming</td><td>8</td></tr>
  </tbody>
</table>
```

14

Default styles, units and text

Default styles

Every browser has its own default styles (margins on body, sizes of headings). A CSS reset or normalize makes the start the same in every browser.

CSS units

Absolute: px. Relative: em (parent font size), rem (root font size), % and vh / vw (viewport). I use rem for font sizes and px for borders.

More text styles

font-family with fallbacks, Google Fonts or @font-face, font-weight, letter-spacing, line-height, text-transform, text-shadow and text-overflow: ellipsis.

More CSS properties

background shorthand, border-radius, box-shadow, overflow, opacity. Small properties that make a page look finished.

15

Advanced selectors

Child and sibling – > (direct child), + (next sibling), ~ (all next siblings)

Pseudo-classes – a state: :hover, :focus, :first-child, :nth-child()

Pseudo-elements – a part: ::before, ::after, ::first-letter, ::marker

Attribute selectors – [type="email"], [href^="https"]

Result – I can style elements without adding many extra classes

selectors.css

```
/* only direct children */
.menu > li { display: inline-block; }

/* every second row of a table */
tr:nth-child(even) { background: #f2f2f2; }

/* input that has focus */
input:focus { outline: 2px solid gold; }

/* add an icon before a link */
a[href^="https"]::before {
  content: "↗ ";
}

/* paragraph right after a heading */
h2 + p { font-size: 1.2rem; }
```

16

Positioning

static – the default, normal flow of the page

relative – moves from its normal place, keeps its space

absolute – taken out of the flow, placed inside the closest positioned parent

fixed – stays in the same place on the screen when scrolling

sticky – normal until you scroll to it, then it sticks

position.css

```
.card {
  position: relative;  /* parent */
}

.badge {
  position: absolute;  /* inside .card */
  top: 10px;
  right: 10px;
}

header {
  position: sticky;    /* my profile nav */
  top: 0;
}
```

17

CSS functions and custom properties

calc() – mix units, e.g. 100% minus the header height

min(), max(), clamp() – sizes that adapt to the screen

Custom properties – variables like --main-color, used with var()

Scope – defined on :root they work everywhere, on an element only inside it

Themes – change a few variables to make a dark theme

functions.css

```
:root {
  --ink: #1b1b22;
  --accent: #f7df1e;
}

h1 {
  /* min 40px, max 68px, grows with screen */
  font-size: clamp(40px, 7vw, 68px);
  color: var(--ink);
}

main {
  height: calc(100vh - 70px);
}

.button {
  background: var(--accent, gold);
}
```

18

Browsers, frameworks, preprocessors

Browser compatibility

Not every browser supports every new feature. I check caniuse.com before I use something new, and test in Chrome, Firefox and Safari.

Frameworks

Bootstrap and Tailwind give ready classes and components, so you write less CSS. But first you need to understand normal CSS.

Preprocessors

Sass, Less and PostCSS add features like nesting, mixins and loops, and compile to normal CSS. Today CSS itself has variables and nesting.

My choice

For these projects I used plain CSS without frameworks, to practice the basics that the course teaches.

19

Forms and form validation

<form> – action and method say where the data is sent

<label for> – connects text with an input, better for accessibility

Input types – email, tel, password, number, date, checkbox, radio

Validation – required, minlength, maxlength, min, max, pattern

Styling – :valid, :invalid and :user-invalid show errors

form.html

```
<form action="/signup" method="post">
  <label for="email">Email</label>
  <input type="email" id="email"
         name="email" required>

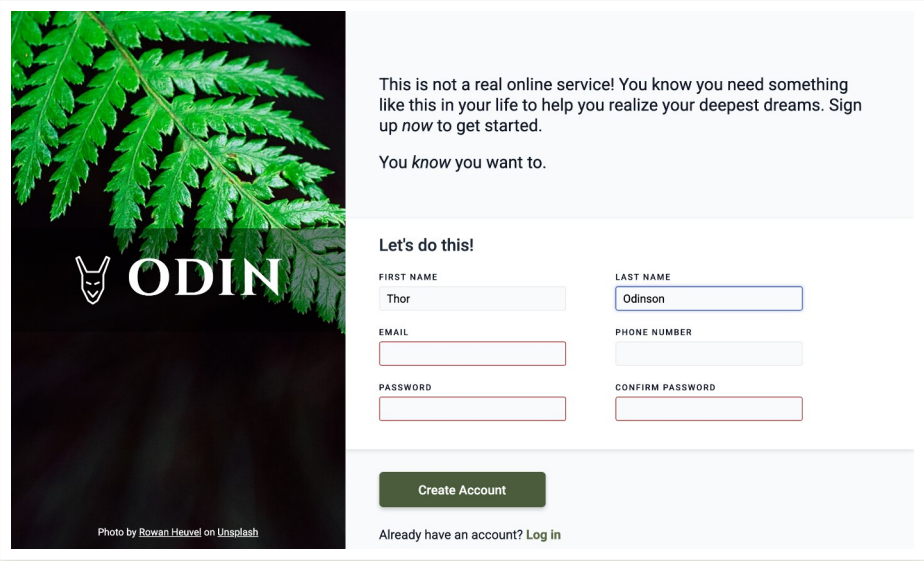
  <label for="password">Password</label>
  <input type="password" id="password"
         name="password" minlength="8"
         required>

  <button>Create Account</button>
</form>

/* red border only after the user typed */
input:user-invalid { border-color: red; }
```

20

Project: Sign-up Form



- Two columns made with Flexbox
- Image side with logo and photo credit
- Labels, input types and required fields
- Red border only after the user interacts
- Small script checks that passwords match

Live ayxan1110.github.io/odin-sign-up-form/

Code github.com/Ayxan1110/odin-sign-up-form

21 CSS Grid

Grid container – display: grid turns children into grid items

Tracks – grid-template-columns / rows, the fr unit, repeat()

Gap – space between rows and columns

Placing items – grid-column: 1 / 3, grid-row, grid-template-areas

Responsive – repeat(auto-fit, minmax(240px, 1fr)) without media queries

grid.css

```
.dashboard {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-rows: auto 1fr;
  min-height: 100vh;
}

.sidebar {
  grid-row: 1 / 3;   /* full height */
}

.cards {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(240px, 1fr));
  gap: 20px;
}
```

22

Flexbox or Grid?

Flexbox: one direction

Good for a row or a column of items: a navigation bar, buttons in a row, centering one thing. The content decides the size.

Grid: two directions

Good for the whole page layout or a gallery of cards, where rows and columns must line up. The layout decides the size.

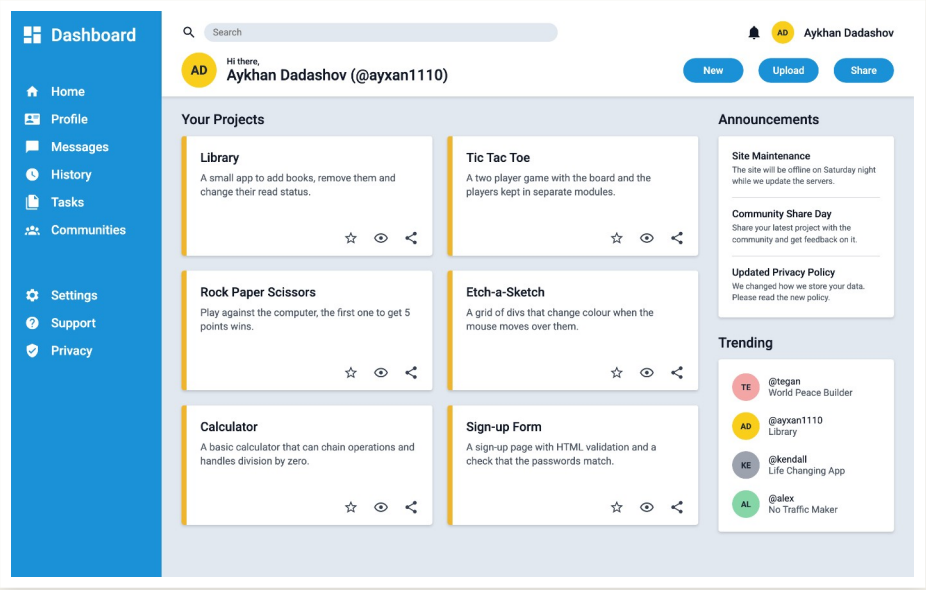
Together

In the Admin Dashboard the page and the cards use Grid, and the header, buttons and items inside a card use Flexbox.

Advanced grid

auto-fill vs auto-fit, minmax(), implicit tracks with grid-auto-rows, and naming lines and areas.

23 Project: Admin Dashboard



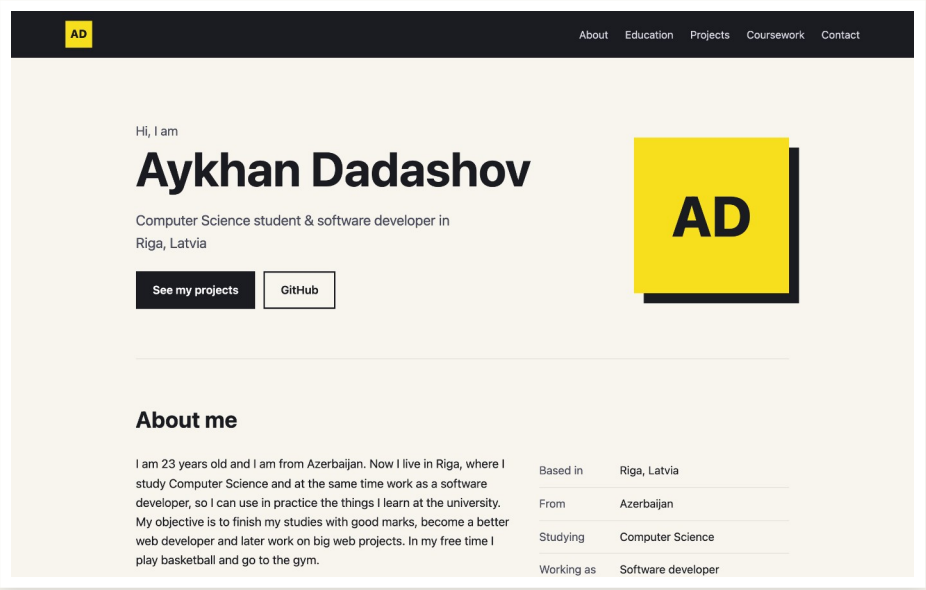
- Page layout made with CSS Grid
- Sidebar with icons (inline SVG)
- Header with search, avatar and buttons
- Project cards in a responsive grid
- Announcements and Trending in the right column

Live ayxan1110.github.io/odin-admin-dashboard/

Code github.com/Ayxan1110/odin-admin-dashboard

24

My profile page on GitHub Pages



- Hosted for free with GitHub Pages
- About me, education and skills
- Cards linking to all my projects
- Table with all Web Technologies tasks
- Sticky nav, CSS variables, clamp(), Grid

Live ayxan1110.github.io/

Code github.com/Ayxan1110/Ayxan1110.github.io

PART 3

JavaScript course

Organizing your JavaScript code: Library and Tic Tac Toe

25 What I covered

1
Introduction: How This Course Will Work

2
Organizing Your JavaScript Code

3
Organizing Code with Objects

4
Object Constructors

5
Project: Library

6
Factory Functions and the Module Pattern

7
Project: Tic Tac Toe

26 Introduction: how the course works

Builds on Foundations – we already know variables, functions, loops and the DOM

Focus on structure – not only "does it work", but how to keep code clean when the project grows

Read, then build – each lesson has articles and knowledge checks, and every few lessons there is a project

Ask for help – the course recommends the Odin community on Discord when you are stuck

14 lessons in "Organizing Your JavaScript Code"

4 projects in the section: Library, Tic Tac Toe, Restaurant Page, Todo List

2 projects done so far: Library and Tic Tac Toe

27 Organizing code with objects

Object literal – groups data that belongs together in one place

Dot or bracket notation – player.name or player["name"] (brackets work with variables)

Methods – functions inside an object, so data and behaviour live together

this – inside a method it points to the object that called it

Result – instead of many separate variables, one object keeps the code organized

objects.js

```
// without objects: loose variables
const playerOneName = "Aykhan";
const playerOneMarker = "X";

// with an object: grouped together
const playerOne = {
  name: "Aykhan",
  marker: "X",
  sayHello() {
    return `Hi, I am ${this.name}`;
  },
};

playerOne.name;         // "Aykhan"
playerOne["marker"];    // "X"
playerOne.sayHello();   // "Hi, I am Aykhan"
```

28

Object constructors and prototypes

Constructor – a normal function we call with new to create many similar objects

new – creates an empty object, sets this to it and returns it

Prototype – every object has one; methods there are shared, not copied

Prototype chain – if a property is not found, JS looks in the prototype

Inheritance – Object.setPrototypeOf(Child.prototype, Parent.prototype)

constructor.js

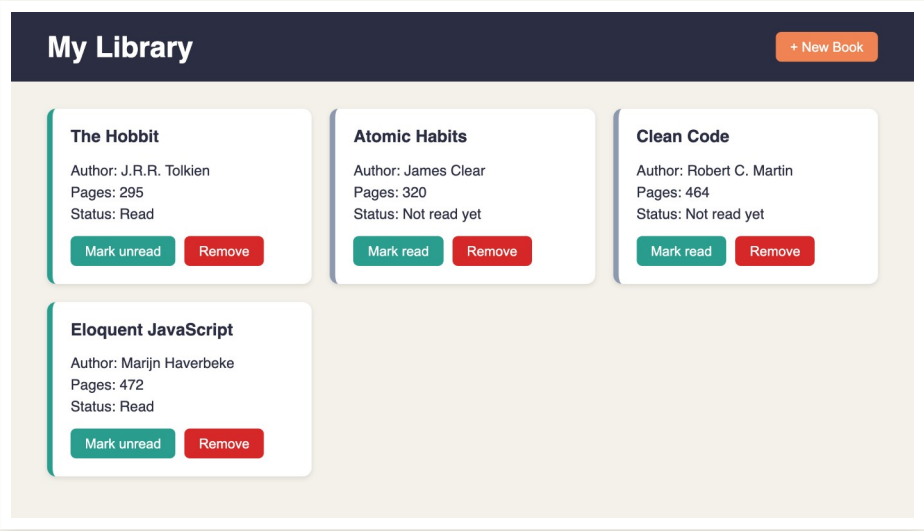
```
function Book(title, author, pages, read) {
  if (!new.target) throw Error("Use new");
  this.title = title;
  this.author = author;
  this.pages = pages;
  this.read = read;
}

// one shared function for all books
Book.prototype.toggleRead = function () {
  this.read = !this.read;
};

const hobbit = new Book("The Hobbit",
  "Tolkien", 295, true);
hobbit.toggleRead(); // read = false
```

29

Project: Library



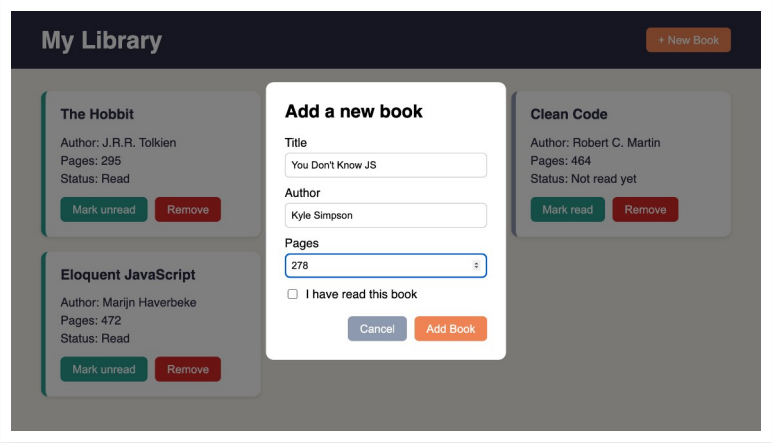
- Shows all books as cards from the myLibrary array
- "New Book" opens a <dialog> with a form
- Remove a book with its button
- Toggle read / not read with a prototype method
- Every book has a unique id from crypto.randomUUID()

Live ayxan1110.github.io/odin-library/

Code github.com/Ayxan1110/odin-library

30

Library: how I built it



The form inside <dialog>, opened with showModal()

1. **Open**
   Click on "New Book" calls dialog.showModal()

2. **Submit**
   preventDefault() stops the page reload, then I read the input values

3. **Store**
   addBookToLibrary() creates new Book(...) and pushes it to myLibrary

4. **Display**
   displayBooks() clears the container and draws a card for every book

5. **Connect**
   Each card has data-id, so Remove finds the right book in the array

31

Factory functions and the module pattern

Scope – variables live inside the function or block where they are declared

Closure – an inner function remembers the variables of its outer function

Factory function – a function that returns an object, no new and no this needed

Private variables – only reachable through the returned functions

Module (IIFE) – a factory that runs one time, for things we need only once

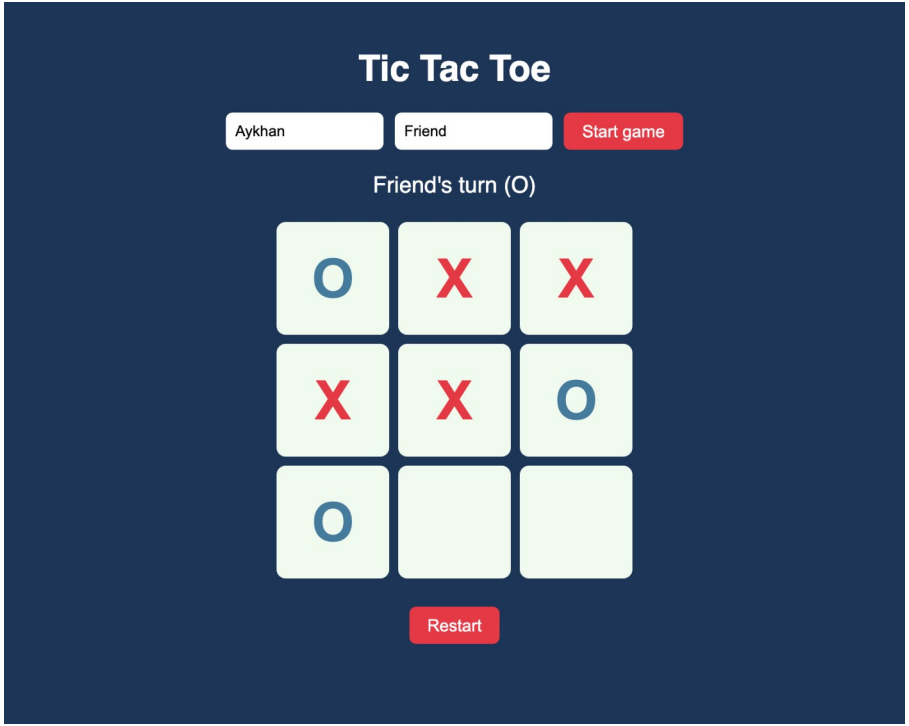
factory.js

```
// factory: we can make many players
function createPlayer(name, mark) {
  return { name, mark };
}
const p1 = createPlayer("Aykhan", "X");

// module: runs once, board stays private
const Gameboard = (function () {
  const board = ["", "", "", "", "",
                 "", "", "", ""];
  const getBoard = () => board;
  const placeMark = (i, mark) => {
    if (board[i] !== "") return false;
    board[i] = mark;
    return true;
  };
  return { getBoard, placeMark };
})();
```

32

Project: Tic Tac Toe

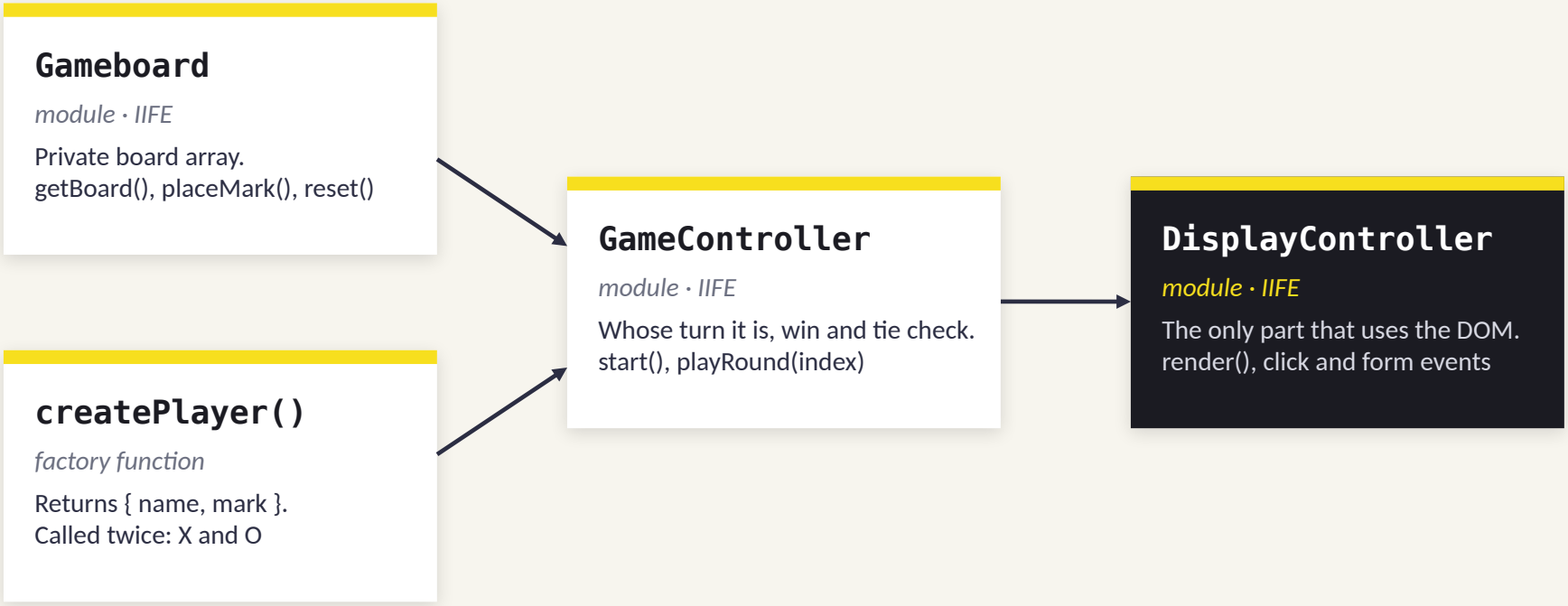


- Two players enter their names
- Click a cell to place X or O
- Taken cells cannot be clicked again
- Detects all 8 winning lines and a tie
- Restart button starts a new round

Live ayxan1110.github.io/odin-tic-tac-toe/

Code github.com/Ayxan1110/odin-tic-tac-toe

Tic Tac Toe: code structure



Game logic works without the DOM, so it can be played in the console

Problems I solved

form.title was empty

A <form> already has a built-in title property, so form.title did not return my input. I used document.querySelector("#title") instead.

Books were duplicated

Every time I added a book, all cards were added again. displayBooks() now clears the container first and then draws the array.

Win on the last move

If someone wins with the 9th move, the game must not say "tie". I check for a winner first and only then for a full board.

Nine click listeners

Instead of adding a listener to every cell, I add one listener on the board and read data-index from the clicked cell (event delegation).

PART 4

Advanced HTML and CSS

Animation, accessibility, responsive design and the Homepage

35

Advanced HTML and CSS: what I covered

Animation

Transforms

Transitions

Keyframes

Accessibility

Introduction to Web Accessibility

WCAG

Semantic HTML

Accessible Colors

Keyboard Navigation

Meaningful Text

WAI-ARIA

Accessibility Auditing

Responsive Design

Introduction to Responsive Design

Natural Responsiveness

Responsive Images

Media Queries

Project: Homepage

36

Transforms and transitions

transform – move, rotate, scale or skew an element without changing the layout

Functions – translate(), rotate(), scale(), skew(), and 3D versions

transition – animates a change between two states, like on :hover

Parts – property, duration, timing function (ease) and delay

Performance – animate transform and opacity, they are cheap for the browser

animation.css

```
.card {
  transition: transform 0.25s ease,
              box-shadow 0.25s ease;
}

/* lift the card on hover */
.card:hover {
  transform: translateY(-6px);
  box-shadow: 0 12px 24px rgb(0 0 0 / 0.15);
}

/* small rotation on the logo */
.logo:hover {
  transform: rotate(-6deg);
}
```

37 Keyframes

@keyframes – define the steps of an animation with from / to or percentages

animation – name, duration, timing, delay, iteration-count, direction

Difference – a transition needs a trigger, keyframes can run on their own and loop

fill-mode – keeps the styles of the first or last keyframe

Use it carefully – too much movement can be distracting

keyframes.css

```css
@keyframes bounce {
  0%   { transform: translateY(0); }
  50%  { transform: translateY(-20px); }
  100% { transform: translateY(0); }
}

.ball {
  animation-name: bounce;
  animation-duration: 1s;
  animation-timing-function: ease-in-out;
  animation-iteration-count: infinite;
}

/* shorthand */
.ball { animation: bounce 1s infinite; }
```

38

Web accessibility and WCAG

Why accessibility

Many people use the web with a screen reader, only the keyboard, zoom or with colour blindness. An accessible site works for all of them, and it is also better for SEO.

WCAG

The Web Content Accessibility Guidelines. Four principles (POUR): Perceivable, Operable, Understandable, Robust, and three levels A, AA and AAA. Most sites aim for AA.

Semantic HTML

Use the right element: <button> for actions, <a> for links, <nav>, <main>, <header>, headings in order. Screen readers understand the page from these elements.

Accessible colours

Text needs enough contrast with the background: at least 4.5:1 for normal text (AA). Never use only colour to show information, like errors.

39

Keyboard navigation and meaningful text

Keyboard – everything must work with Tab, Enter and Space

Focus styles – never remove the outline without a better :focus-visible style

Skip link – lets keyboard users jump over the navigation

Meaningful text – links like "View Library code" instead of "click here"

alt text – describe what the image shows; empty alt for decoration

a11y.html

```
<!-- first element in the body -->
<a class="skip-link" href="#main">Skip to
content</a>

.skip-link {
  position: absolute;
  top: -100px;          /* hidden above */
}
.skip-link:focus {
  top: 0;               /* visible on Tab */
}

a:focus-visible {
  outline: 3px solid var(--accent);
}

<img src="library.jpg"
     alt="Library app with book cards">
```

40

WAI-ARIA and accessibility auditing

What ARIA is

Attributes that give extra information to screen readers: role, aria-label, aria-labelledby, aria-hidden, aria-expanded, aria-live.

First rule of ARIA

Don't use ARIA if a native HTML element can do it. A <button> is better than a . No ARIA is better than bad ARIA.

In my Homepage

aria-labelledby connects each section to its heading, aria-label names the navigation, and aria-hidden hides decorative SVG icons.

Auditing

Lighthouse in Chrome DevTools, the axe extension and WAVE find many problems automatically. Then I test by hand with the keyboard and zoom.

41 Natural responsiveness and responsive images

Responsive design – one website that works on phone, tablet and desktop

Natural responsiveness – avoid fixed widths; use max-width, %, Flexbox and Grid

Viewport meta tag – required, or phones show a zoomed-out desktop page

Images – max-width: 100% so they shrink with the container

srcset and sizes – the browser downloads the right image size

responsive.html

```
<meta name="viewport" content="width=device-
width">

img { max-width: 100%; height: auto; }

/* cards wrap by themselves, no media query */
.projects {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(280px, 1fr));
}

<img src="library-800.jpg"
     srcset="library-400.jpg 400w,
             library-800.jpg 800w"
     sizes="(min-width: 960px) 340px, 90vw"
     alt="Library app">
```

42 Media queries

@media – apply CSS only when a condition is true, like a minimum width

Mobile first – write phone styles first, then add min-width queries

Breakpoints – choose them where the design breaks, not for specific devices

Other features – prefers-color-scheme, prefers-reduced-motion, print

My breakpoints – 640px (tablet) and 960px (desktop)

media.css

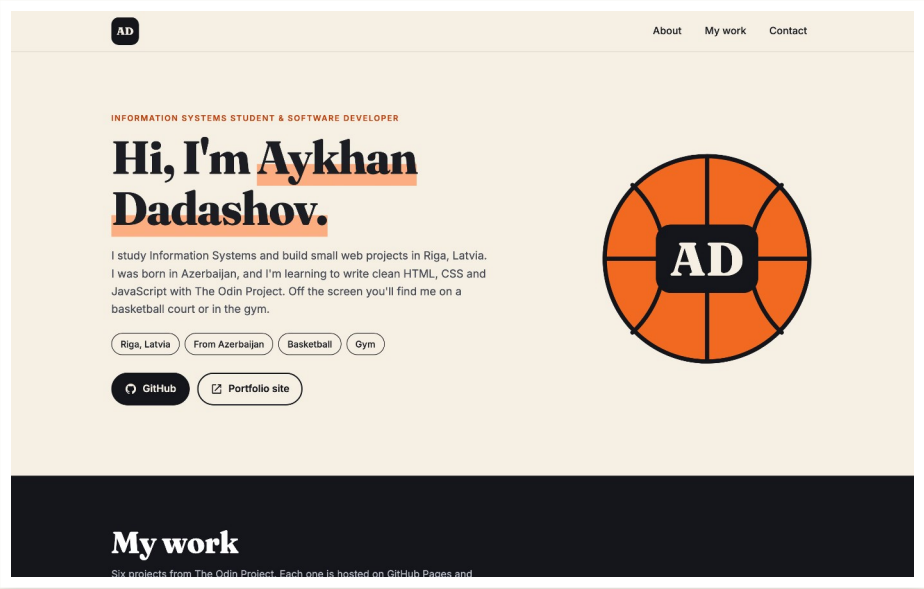
```
/* phone first: no media query */
.hero-inner { display: grid; }

/* tablet and up */
@media (min-width: 640px) {
  .hero { padding: 4rem 0; }
}

/* desktop: text left, portrait right */
@media (min-width: 960px) {
  .hero-inner {
    grid-template-columns: 1fr 340px;
  }
}

@media (prefers-reduced-motion: reduce) {
  * { transition: none !important; }
}
```

43 Project: Homepage



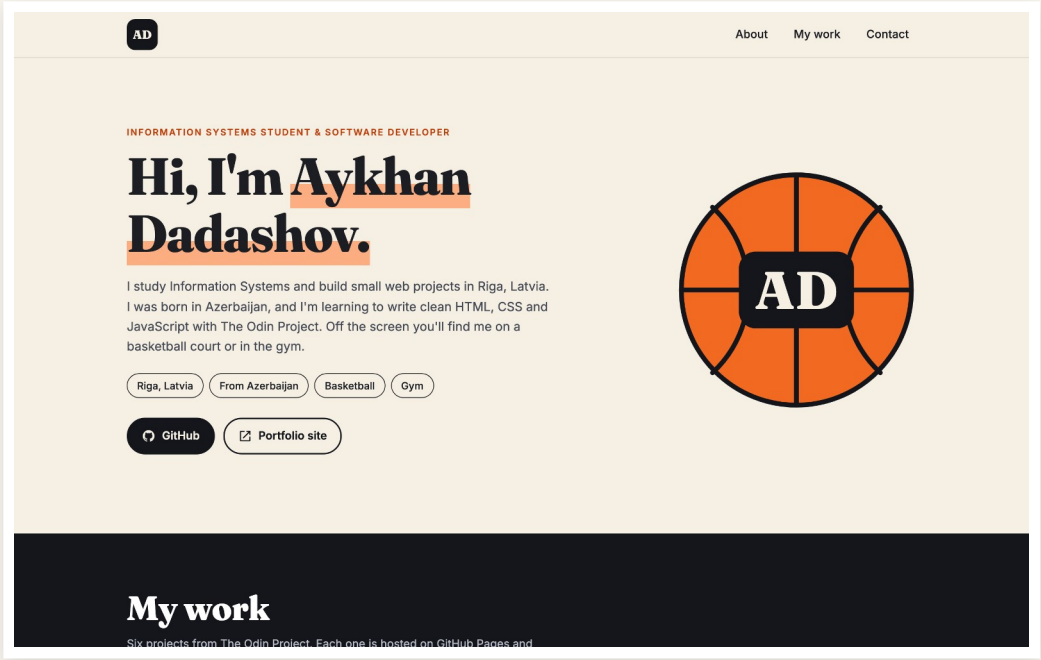
- Personal homepage with my real projects
- Mobile first, breakpoints at 640px and 960px
- Responsive images with srcset and sizes
- Hover transitions on cards and the logo
- Skip link, focus styles, reduced motion

Live ayxan1110.github.io/odin-homepage/

Code github.com/Ayxan1110/odin-homepage

Homepage on phone and desktop





Same HTML, different layout: one column on the phone, two columns on desktop

PART 5

Project idea: CareerBridge

An AI career coach for international students in Latvia

45

CareerBridge in one slide

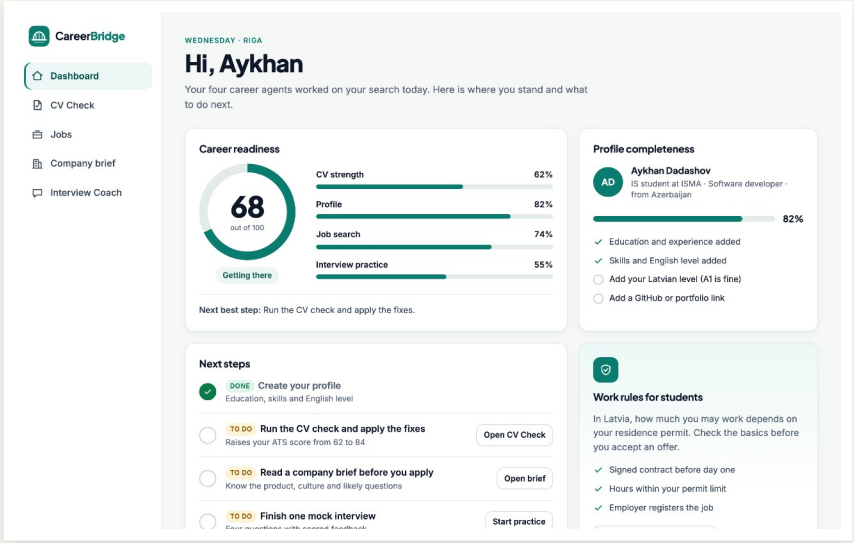
Problem – only 43% of non-EU graduates stayed in Latvia to work in 2024 (58% in 2022)

Solution – four AI agents: CV check, job scout, company brief, interview coach

Human in control – the student approves every CV and application

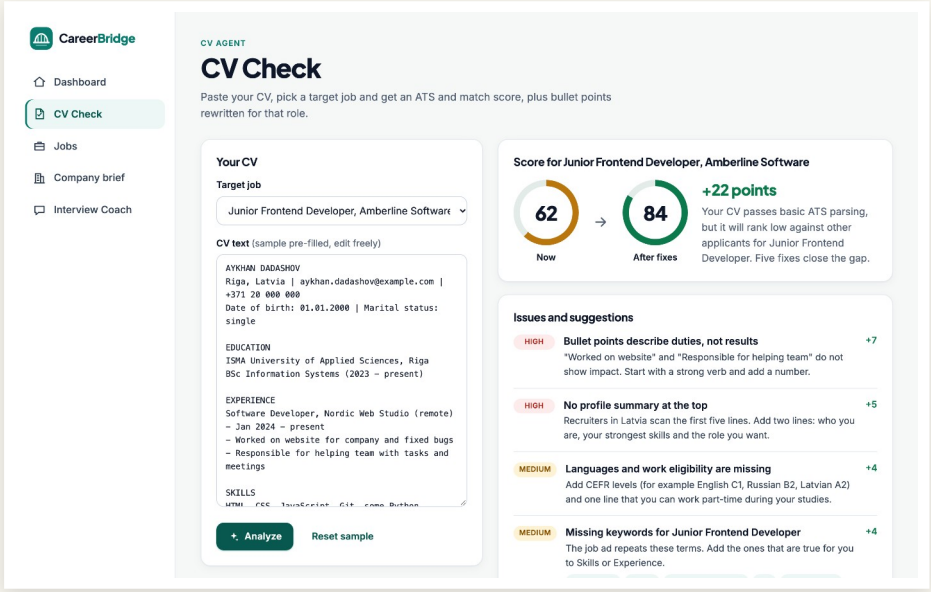
Model – free for students, Pro €4.99 / month, licences for universities, job posts for employers

Based on – 7 GitHub repos I reviewed, like awesome-llm-apps, STORM and ai-employees



Prototype ayxan1110.github.io/careerbridge-prototype/

CareerBridge: clickable prototype



- Dashboard with a career readiness score
- CV Check: score 62 → 84 with fixes
- Jobs with match % and real filters
- Company brief with citations
- Interview Coach with feedback

Live ayxan1110.github.io/careerbridge-prototype/

Code github.com/Ayxan1110/careerbridge-prototype

47 All links

Project	Code on GitHub
Rock Paper Scissors	github.com/Ayxan1110/odin-rock-paper-scissors
Etch-a-Sketch	github.com/Ayxan1110/odin-etch-a-sketch
Calculator	github.com/Ayxan1110/odin-calculator
Sign-up Form	github.com/Ayxan1110/odin-sign-up-form
Admin Dashboard	github.com/Ayxan1110/odin-admin-dashboard
Library	github.com/Ayxan1110/odin-library
Tic Tac Toe	github.com/Ayxan1110/odin-tic-tac-toe
Homepage	github.com/Ayxan1110/odin-homepage
CareerBridge	github.com/Ayxan1110/careerbridge-prototype
Profile with everything	ayxan1110.github.io/